HULL TACTICAL MARKET PREDICTION: A ROBUST STACKING ENSEMBLE APPROACH

Luo TianCheng **Wan Qianhua** **Hng Cherng Khai** **Chan Kee Qing**

Chen Yike

ABSTRACT

Our Project Group predict forward excess returns of the S&P 500 using financial indicators from Hull Tactical. Through comparative analysis of several models, we demonstrate that single-model architectures suffer from severe overfitting (stability gaps of -88.7% and -78.3%). The Stacking Ensemble achieves perfect stability (0.0% gap) with a score of 0.460, aligning with the Kaggle Public Leaderboard score of 0.461. The ensemble's Ridge Regression meta-learner smooths predictions, effectively extracting stationary signals from noisy financial data. Our results highlight that variance management and model stability are more critical than aggressive signal mining in volatility-adjusted regimes.

1 INTRODUCTION

In this study, we developed a predictive modeling framework for short-horizon financial returns, aiming to balance predictive accuracy with robust risk control under realistic market conditions. Financial time-series data were characterized by non-stationarity, noise, and nonlinear dynamics, making reliable forecasting particularly challenging. To address these issues, we adopted a hybrid approach that combined linear regularization with modern gradient-boosting methods and ensemble learning.

Our framework employed Ridge Regression, LightGBM, and XGBoost as complementary base learners within a two-layer Stacking Ensemble. Ridge Regression captured stable linear structures, while LightGBM and XGBoost modeled nonlinear interactions and regime shifts. Using Ridge Regression as a meta-learner allowed the ensemble to integrate diverse predictive signals and reduce variance through structured aggregation.

To ensure realistic evaluation, all experiments followed strict temporal ordering with an 80–20 time-series split. The inference pipeline was implemented using the Kaggle Evaluation API.

2 DATA EXPLORATION & FEATURE ENGINEERING

The Hull Tactical dataset provides approximately 9,000 daily observations of diverse financial signals, including macroeconomic indicators, sentiment measures, technical factors, price-based features, and forward returns. Financial time-series data presents several unique challenges:

- Time-dependent structure
- Missing and irregularly spaced features
- Non-stationarity across market cycles
- Severe noise in daily return data
- High multicollinearity among factors
- Risk of lookahead bias

A robust feature engineering pipeline was therefore essential prior to model training.

2.1 Overview and Preprocessing

The dataset provided for the Hull Tactical competition consists of historical market data spanning several decades, containing hundreds of features categorized into market dynamics (M^*), macro-economic indicators (E^*), and sentiment variables (S^*) (Hull Tactical Asset Allocation, 2024). The primary challenge in this dataset is the presence of high-dimensional noise and missing values in early historical observations.

Our initial exploration revealed that the raw data contained non-informative identifiers (`row_id`, `date_id`) which were removed to prevent spurious correlations. The target variable, `forward_returns`, represents the return from buying the S&P 500 and selling it the following day. To prepare the data for our stacking ensemble, we standardized all input features using Z-score normalization (StandardScaler), ensuring that the varying scales of interest rates and price metrics did not bias the gradient-based optimization in our downstream models.

2.2 Advanced Missing Value Imputation

Given the financial nature of the data, simple mean or median imputation can destroy volatility signals inherent in market crashes or rallies. Instead of static imputation, we employed a multivariate approach using the Iterative Imputer (MICE algorithm).

We configured the imputer with a Ridge Regression estimator ($\alpha = 1.0$) to model each feature with missing values as a function of other features. This method preserves the covariance structure between market indicators, allowing the model to infer missing macro-economic states based on available price and volatility metrics. This represents a significant improvement over baseline methods, as it reconstructs the market context of missing data points rather than simply filling gaps with central tendencies.

2.3 Anomaly Detection via Isolation Forest

Financial time series are characterized by "fat tails"—extreme market events that are statistically rare but highly informative. Standard outlier removal (e.g., deleting rows beyond 3 standard deviations) would be detrimental, as it would remove the very crash scenarios our model needs to learn to manage risk effectively.

To address this, we implemented an unsupervised anomaly detection mechanism using an Isolation Forest (contamination set to 0.1). This algorithm isolates observations by randomly selecting a feature and then randomly selecting a split value between the maximum and minimum values of the selected feature.

Rather than discarding anomalies, we adopted a "feature augmentation and preservation" strategy:

1. **Explicit Feature Generation**: We generated two new features: a binary flag `is_anomaly` and a continuous `anomaly_score`. These allow the downstream regressor to explicitly recognize and treat high-stress market regimes differently from normal trading days.
2. **Conditional Winsorization**: We applied Winsorization only to the samples flagged as anomalies. For these rows, feature values falling outside the 5th and 95th percentiles of the normal data distribution were clipped. This reduces the leverage of extreme noise while preserving the directionality of the market signal.

2.4 Intelligent Feature Selection

To mitigate the curse of dimensionality and reduce overfitting, we applied a three-stage feature selection pipeline:

1. **Variance Thresholding**: We removed features with variance below 0.001, eliminating variables that remain near-constant and offer no predictive signal.
2. **Correlation Filtering**: We calculated the pairwise correlation matrix and removed features with a Pearson correlation coefficient greater than 0.98 to reduce multicollinearity, which is particularly harmful to the stability of our linear base learners (Ridge Regression).

3. **Target-Aware Selection**: Finally, we employed SelectKBest using the f-regression score to retain the top k features (where $k \geq 60$) that showed the strongest statistical relationship with `forward_returns`.

This pipeline reduced the feature space to a robust subset, balancing model complexity with information retention.

3 Model Selection & Training

3.1 LightGBM

In this competition, the feature space is high-dimensional and enriched with macro-economic, volatility, and sentiment indicators that are often sparse and nonlinear. Together with millions of rolling-window observations, this makes computational efficiency essential. Gradient-boosting tree models are well suited to such settings, and LightGBM in particular is widely adopted for its efficiency and ability to exploit sparsity and complex interactions. We therefore use LightGBM as our initial baseline and first experiment with a pure LightGBM setup.

3.1.1 LightGBM: Efficiency-First Processing for Large-Scale Data

LightGBM, introduced by Microsoft, focuses on enhancing training speed and reducing memory consumption through two key innovations: GOSS (Gradient-based One-Side Sampling) and EFB (Exclusive Feature Bundling) (Ke et al., 2017).

Objective Function LightGBM optimizes a regularized objective that combines a training loss with tree complexity, and evaluates splits using second-order gradient information.

Gradient-based One-Side Sampling (GOSS) GOSS retains all instances with large gradients and randomly samples only a subset of small-gradient instances, reducing computation while preserving predictive power.

EFB (Exclusive Feature Bundling) For high-dimensional sparse data, EFB bundles mutually exclusive features into a single new feature. By adding offsets to original feature values, it drastically reduces feature dimensionality without information loss.

Leaf-wise (Best-first) Growth Strategy Unlike XGBoost's level-wise strategy, LightGBM adopts a leaf-wise (best-first) growth, repeatedly splitting the leaf that yields the largest second-order gain. The split gain is computed as:

$$Gain = \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}, \tag{1}$$

where G and H denote aggregated first- and second-order gradients. This criterion lets LightGBM prioritize the most informative leaf at each step.

LightGBM not only trains faster with lower memory usage but its leaf-wise growth enables more aggressive fitting of complex patterns, which can be useful in low signal-to-noise settings and sparse or irregular temporal features. However, this aggressiveness—together with GOSS's focus on high-gradient instances—also increases overfitting risk on small or noisy datasets, requiring careful regularization.

3.1.2 Results

Using the full training period, the pure LightGBM model achieved a public score of 4.12, whereas using an internal 80/20 split it reached only 0.456. Since the Kaggle evaluation is based on the final portion of the training timeline, this discrepancy indicates that LightGBM severely overfits and exhibits substantial instability in out-of-sample testing. These results suggest that a pure LightGBM approach is insufficient for this task and that more advanced modeling strategies are required.

3.2 Random Forest Regressor

To complement the boosting approach, we employ a Random Forest Regressor. While decision trees are non-parametric models that allow data to determine the form of fitted functions, single decision trees suffer from high variance and a tendency to overfit the training data, resulting in poor generalization on testing data (Zhou, 2025b, Lecture 8, p. 5).

We choose Random Forest to mitigate this high variance through the "wisdom of the crowd" philosophy. The model relies on two specific mechanisms (Zhou, 2025b, Lecture 8, p. 50):

3.2.1 Bootstrap Aggregating (Bagging)

The Random Forest follows the Bagging process, which introduces data-level randomness.

- **Step 1**: Generate multiple diverse training sets by sampling records from the original data with replacement (bootstrapping).
- **Step 2**: Fit an overgrown decision tree to each of these resampled training datasets.

By aggregating the predictions of these diverse trees (via averaging for regression), we reduce the variance associated with any single model.

3.2.2 Split-Attribute Randomization

Bagging alone is often insufficient because it can produce correlated trees if strong predictors dominate every split. To address this, Random Forest introduces randomness at the attribute level. Each time a split is performed, the search for the best split attribute is limited to a random subset of the total attributes (Zhou, 2025b, Lecture 8, pp. 56, 58, 59).

3.2.3 Justification for Model Selection

We selected Random Forest because it is robust to outliers and noise in financial time-series data. While single trees are prone to overfitting, the ensemble nature of Random Forest effectively manages the bias-variance tradeoff, providing a lower-variance solution that generalizes better to unseen market regimes.

3.3 Ridge Regression and Stacking Ensemble

We also employed Ridge Regression as the core linear foundation, augmented by XGBoost and LightGBM to capture complex non-linearities.

3.3.1 Model Framework

The predictive framework adopted a stacking ensemble composed of two layers. Stacking allowed the model to integrate heterogeneous learners and exploit complementary strengths. Formally, let the base models produce predictions $\hat{y}_i^{(1)}, \hat{y}_i^{(2)}, \hat{y}_i^{(3)}$, and let the meta-learner be f_{meta}. The final prediction was $\hat{y}_i = f_{meta}(\hat{y}_i^{(1)}, \hat{y}_i^{(2)}, \hat{y}_i^{(3)})$. This structure increases robustness and reduces the variance of individual models.

First Layer: Base Models The first layer contained three distinct base learners, each capturing a different class of data-generating patterns.

- **Ridge Regression (Linear Model)**: Ridge Regression served as the linear component, suitable for capturing additive and approximately linear relationships. The estimator minimized $\hat{\beta} = \arg\min_\beta(\|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2)$, with closed-form solution $\hat{\beta} = (X^\top X + \lambda I)^{-1} X^\top y$. The L_2 penalty improved stability and mitigated overfitting (Zhou, 2025a, Lecture 5, p. 62).

  Ridge Regression serves as the linear foundation of our first layer. From a deterministic perspective, we seek to minimize the sum of squared errors. However, to prevent overfitting, where the model fits the noise rather than the signal, we want to introduce an l_2

regularization term. The objective function is formulated as:

$$J(w) = \sum_{i=1}^{m}(y_i - w^\top x_i)^2 + \lambda||w||_2^2 \tag{2}$$

From a probabilistic perspective, this is equivalent to finding the Maximum A Posteriori (MAP) estimation assuming the parameters w follow a zero-mean Gaussian prior distribution $p(w) = \mathcal{N}(0, \tau^2 I)$. This Gaussian prior encourages the weights to be small ("weight decay"), resulting in a smoother curve less sensitive to noise. Crucially, the closed-form solution $\hat{w} = (X^\top X + \lambda I)^{-1} X^\top y$ ensures that the matrix is invertible, solving potential multicollinearity issues inherent in correlated financial indicators.

- **LightGBM (Gradient Boosting Decision Trees)**: LightGBM modeled complex non-linear relationships through leaf-wise gradient boosting (Ke et al., 2017). At boosting iteration t, it constructed a tree f_t to minimize the second-order Taylor expansion:

$$L^{(t)} \approx \sum_{i=1}^{n}(g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2) + \Omega(f_t), \tag{3}$$

where g_i and h_i were first- and second-order derivatives of the loss. LightGBM further accelerated training using histogram binning and Gradient-based One-Side Sampling (GOSS).

- **XGBoost (Regularized Gradient Boosting)**: XGBoost served as another non-linear base learner with structural regularization (Chen & Guestrin, 2016). Its objective similarly minimized

$$L^{(t)} = \sum_{i=1}^{n}(g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2) + \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2, \tag{4}$$

where T was the number of leaves. XGBoost's explicit regularization and shrinkage complemented LightGBM and increased the diversity of the ensemble.

Second Layer: Meta-Learner Ridge Regression as Meta-Learner, it took the predictions of the three base models as inputs. Let $Z = [\hat{y}^{(1)}\hat{y}^{(2)}\hat{y}^{(3)}]$. Ridge Regression then solved $\hat{\theta} = \arg\min_\theta \|y - Z\theta\|_2^2 + \alpha\|\theta\|_2^2$, producing the final prediction $\hat{y} = Z\hat{\theta}$. This meta-learning step effectively fused the complementary strengths of the linear and tree-based models.

3.4 Justification for Model Selection

Our choice of architecture is driven by the inherent trade-offs between parametric and non-parametric models:

1. **Bias-Variance Management**: Ridge Regression (parametric) has high bias but low variance, providing stability. LightGBM and Random Forest (non-parametric) have low bias but high variance, allowing them to capture complex structures. The Stacking Ensemble balances these by using the "wisdom of the crowd" to reduce overall variance.
2. **Robustness to Noise**: Financial data contains significant outliers. Standard linear regression minimizes residuals quadratically, meaning outliers can severely skew the model. By using Ridge Regression (Gaussian prior) and Random Forest (bagging), we introduce regularization mechanisms that prevent the model from over-reacting to specific noisy data points.
3. **Handling Sparsity**: The inclusion of LightGBM with EFB ensures that our high-dimensional, sparse feature set is handled efficiently without identifying zero values as informative splits, a common inefficiency in standard histogram-based algorithms.

3.4.1 Hyperparameter Optimization

For Ridge Regression, we performed a grid search over regularization strengths: $\lambda \in \{\lambda_1, \lambda_2, \ldots, \lambda_K\}$, selecting the value with the lowest validation error. For the boosting models

(LightGBM and XGBoost), we used early stopping to automatically select the optimal number of boosting iterations. At iteration t, the model continued if $L_{valid}^{(t)} < L_{valid}^{(t-s)}$, and stopped otherwise, where s was the patience parameter. This prevented overfitting to the training sequence and improved generalization.

4 Evaluation and Comparative Analysis

4.1 Evaluation API Usage

To simulate a realistic quantitative trading environment and avoid look-ahead bias, we strictly followed the Kaggle competition protocol and used the `kaggle_evaluation` library for inference. A `predict(test_batch)` function was implemented to receive real-time data batches streamed by the `DefaultInferenceServer`. For each batch, the system reproduced the exact training-stage preprocessing (feature alignment and missing-value handling), applied the stacked ensemble model to generate predictions, and returned them through `DefaultInferenceServer.serve()`. Before submission, we used `run_local_gateway` to emulate the full API data pipeline locally and verify the robustness of the inference workflow.

4.2 Comparative Results

To rigorously assess model stability, we evaluated three distinct architectures under two conditions:

1. **Full Training (Leakage Test)**: Training on the complete dataset, which overlaps with the Kaggle Public Leaderboard period. High scores here indicate memorization of noise.
2. **Time-Series Split (Out of Sample Testing)**: Training on the initial 80% chronologically and testing on the unseen final 20%. This strictly mimics a real-world production environment where the model must predict future data.

Our experiment revealed a contrast in stability between the architectures. The standalone models (LightBGM and Random Forest Regressor) showed clear signs of overfitting. They achieved high scores of 4.0 and 2.0 when trained on a full timeline, essentially memorizing the data. However, these performances dropped during the out-of-sample test, dropping to 0.452 and 0.434 respectively.

In contrast, the Stacking Ensemble (XGB+LGB+Ridge) improved regularization significantly. By smoothing out the predictions of the base learners, it eliminated the overfitting issue, holding a stable score of 0.46 metric in both Full Training (Leakage Test) and Time-Series Split (Out of Sample Testing). This confirms that the Stacking Ensemble is modelling the underlying signal rather than the noise, making it more suitable candidate for real financial setting.

4.3 Discussion: Performance Analysis

4.3.1 The Bias-Variance Tradeoff and Overfitting

The experiment results highlight a classic issue in non-parametric learning which is the high variance associated with complex tree-based models. In fact, single decision trees are non-parametric models that do not rely on strong assumptions regarding the shape of the relationship between the variables. Instead, the data is allowed to speak for themselves in determining the form of the fitted functions (Zhou, 2025b, Lecture 8, p. 5). This flexibility allows them to grow deep and overfit to the training data, leading to a highly non-smooth prediction of the constructed trees (Zhou, 2025b, Lecture 8, p. 40).

Our standalone LightBGM and Random Forest Regressor exhibited precisely this behaviour. Achieving a score of 4.0 or 2.0 on the full training set indicates that the trees grew too deep, effectively isolating specific noise points rather than learning generalizable rules. While Random Forest Regressor (ensemble models of decision trees) are designed to reduce this variance by introducing randomness into both the data-level and attribute level, our results suggest that without strict pruning or depth constraints, even bagged ensembles can suffer from overfitting in high-noise financial environments (Zhou, 2025b, Lecture 8, pp. 50, 60).

4.3.2 ALGORITHMIC AGGRESSION: LIGHTGBM'S LEAF-WISE GROWTH

The extreme overfitting observed in the standalone LightGBM model can be attributed to its specific architectural choices, which differ from standard Gradient Boosting Decisiton Tree implementations. LightGBM employs a leaf-wise tree growth strategy, which tends to be more aggressive in minimizing loss compared to traditional level-wise growth (Ke et al., 2017, p. 6).

Additionally, LightGBM utilizes Gradient-based One-Side Sampling (GOSS) to speed up training (Ke et al., 2017, p. 1). GOSS operates on the premise that data instances with larger gradients are "under-trained" and thus contribute more to information gain. Consequently, the algorithm retains all instances with large gradients while randomly sampling those with small gradients (Ke et al., 2017, p. 2). While efficient, this approach carries a risk in financial modeling: "large gradients" often correspond to market anomalies or non-stationary noise. By prioritizing these instances, the model effectively focused its learning capacity on fitting the noise, leading to the inflated training scores we observed.

4.3.3 REGULARIZATION VIA STACKING ENSEMBLE

The Stacking Ensemble (XGB+LGB+Ridge) successfully addressed the overfitting by imposing a parametric constraint on the non-parametric base learners. While the base models provided low-bias, high-variance predictions, the Ridge Regression meta-learner smoothed these outputs. By enforcing a linear structure on the final prediction, the ensemble reduced the noise memorized by the aggressive leaf-wise growth of LightGBM and the deep trees of the Random Forest. This validates the ensemble theory that combining diverse models can stabilize predictions, effectively extracting a stationary signal from noisy financial data.

4.3.4 RISK-ADJUSTED SUPERIORITY: AVOIDING THE VOLATILITY PENALTY

The competition's custom evaluation metric employs a specific Volatility-Adjusted Sharpe Ratio, which introduces a penalty term `vol_penalty` into the denominator of the final score. Crucially, this penalty is triggered when the strategy's volatility exceeds the market's volatility by more than 20% (defined in the scoring function as `strategy_volatility / market_volatility - 1.2`).

The standalone models (LightGBM and Random Forest), in their attempt to overfit idiosyncratic noise, produced highly erratic weight predictions. This variance in position sizing directly inflated the standard deviation of the strategy returns (`strategy_std`), likely pushing the models into the penalty zone where `vol_penalty > 1`.

In contrast, the Stacking Ensemble leveraged the central limit theorem. By averaging the distinct signals from the linear Ridge meta-learner and the non-linear tree base models, the ensemble naturally reduces the variance of the final prediction y_{pred}. This variance reduction directly boosts the Volatility-Adjusted Sharpe Ratio, making the Stacking Ensemble the best choice despite the raw complexity of the single trees.

4.4 RISK MANAGEMENT STRATEGIES: BALANCING RETURN AND RISK

Our strategy aimed not only to maximize predictive performance but also to stabilize risk across market regimes. Three mechanisms were employed:

- **Anomaly-Aware Risk Control**: We applied Isolation Forest to detect anomalous market states. Instead of removing detected outliers, we used Winsorization at the 5% and 95% quantiles. This preserved structural patterns from extreme conditions while preventing noisy tail events from dominating the model.
- **Conservative Weight Mapping**: When mapping predicted excess returns into portfolio weights in [0,2], we used an enhanced sigmoid transformation. This mapping remained flat around zero (neutral exposure) and only amplified weights when predictions showed strong confidence, reducing unnecessary turnover and downside risk.
- **Ensemble Stability**: To address the high variance characteristic of non-parametric learners, we employed a stacking ensemble of three heterogeneous models, which effectively

smoothed the variance or prediction surface and reduced potential overfitting. The Ridge meta-learner performed a regularized weighted combination of base predictions, reducing volatility and improving robustness on unseen data.

4.5 Final Model Selection

Based on the comparative analysis, we selected the Stacking Ensemble as our final model for the Hull Tactical competition. The primary criterion for selection was stability in out-of-sample performance rather than raw training metrics. While the pure LightGBM model achieved a very high score on the full training set, its performance degraded by 88% on the test split (Sharpe: 0.452). A similar pattern of overfitting was observed in the Random Forest Regressor, which plummeted from a training score of 2.0 to 0.434.

In contrast, the Stacking Ensemble demonstrated exceptional robustness. It achieved an internal test score of 0.460, which aligned almost perfectly with the 0.461 score obtained on the Kaggle Public Leaderboard. This near-zero generalization gap confirms that the ensemble successfully mitigated noise and captured stationary market signals. By effectively balancing return maximization with the competition's volatility constraints, the Stacking Ensemble proved to be the superior architecture for risk-adjusted market prediction.

5 Conclusion

In this project, we aimed to predict the forward excess returns of the S&P 500 using a comprehensive dataset of financial indicators, addressing the challenge of extracting signal from a high-noise environment. Our approach moved beyond standard modeling by implementing a robust pipeline that prioritized stability over raw complexity.

The comparative analysis demonstrated that while single-model architectures like LightGBM are computationally efficient, they are prone to severe overfitting in financial time-series contexts. By contrast, the Stacking Ensemble proved to be the optimal solution. By integrating linear regularization with non-linear tree-based learning, and employing strict risk control mechanisms like MICE imputation and anomaly-aware Winsorization, we successfully constructed a strategy that generalizes well to unseen data.

The alignment between our internal test results and the Kaggle Public Leaderboard confirms that our validation strategy mimicked real-world conditions effectively. Ultimately, this project highlights that in volatility-adjusted regimes, the key to generating alpha lies not in aggressive signal mining, but in the careful management of variance and model stability. Future work could further enhance this framework by incorporating deep learning architectures, such as LSTMs, to better capture long-term temporal dependencies in market cycles.

References

Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. ACM, 2016. doi: 10.1145/2939672.2939785.

Hull Tactical Asset Allocation. Hull tactical – market prediction. Kaggle, 2024. URL `https://www.kaggle.com/competitions/hull-tactical-market-prediction`.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pp. 3146–3154, 2017.

J. Zhou. Dda3020 machine learning: Lecture 05 linear regression [course notes], 2025a.

J. Zhou. Dda3020 machine learning: Lecture 08 tree-based methods [course notes], 2025b.